Universidade do Minho

Ano Letivo: 2025/26

Databases

Relational Algebra



Relational Algebra

Introduction

Between 1969 and 1971, Edgar Codd published two fundamental papers in which he introduced the relational data model, as well as the languages for manipulating that data: Relational Algebra and Relational Calculus.

Although the relational model was extremely important, the greatest emphasis at the time was given to practical data manipulation languages. These made it possible, for the first time, to work with information based solely on its logical characteristics, without depending on how the data was physically stored.

In 1970, Codd established the foundations of Relational Algebra, which became essential for the development of modern databases.

Relational Algebra

What is it?

Relational Algebra is one of the two formal query languages associated with the Relational Model. It is a theoretical language that defines a set of operations that can be applied to one or more relations (tables), producing a new relation as a result, without modifying the original relations.

Queries in Relational Algebra are constructed from a set of operators, which allow data to be manipulated and combined in different ways.

Relational Algebra

Operations

Relational Algebra operations act on relationships, that is, they use one or two tables as input to generate a new relationship. This new relationship can in turn be used as input in subsequent operations.

The resolution of a query can be done in a phased, step-by-step manner. This allows you to test intermediate or partial results, making it easier to experiment and optimize the best way to obtain the final result of the interrogation.

In relational algebra, we can distinguish between unary operations (operations that occur only on a single relation) and binary operations (operations that occur on two relations).

Relational Algebra

Operators

UNARY OPERATIONS

σ (“sigma”)	selection
π (“pi”)	projection
$\leftarrow$	attribution
δ (“delta”) ρ (“rho”)	renaming
τ (“tau”)	ordering
γ (“gama”)	aggregation

BINARY OPERATIONS

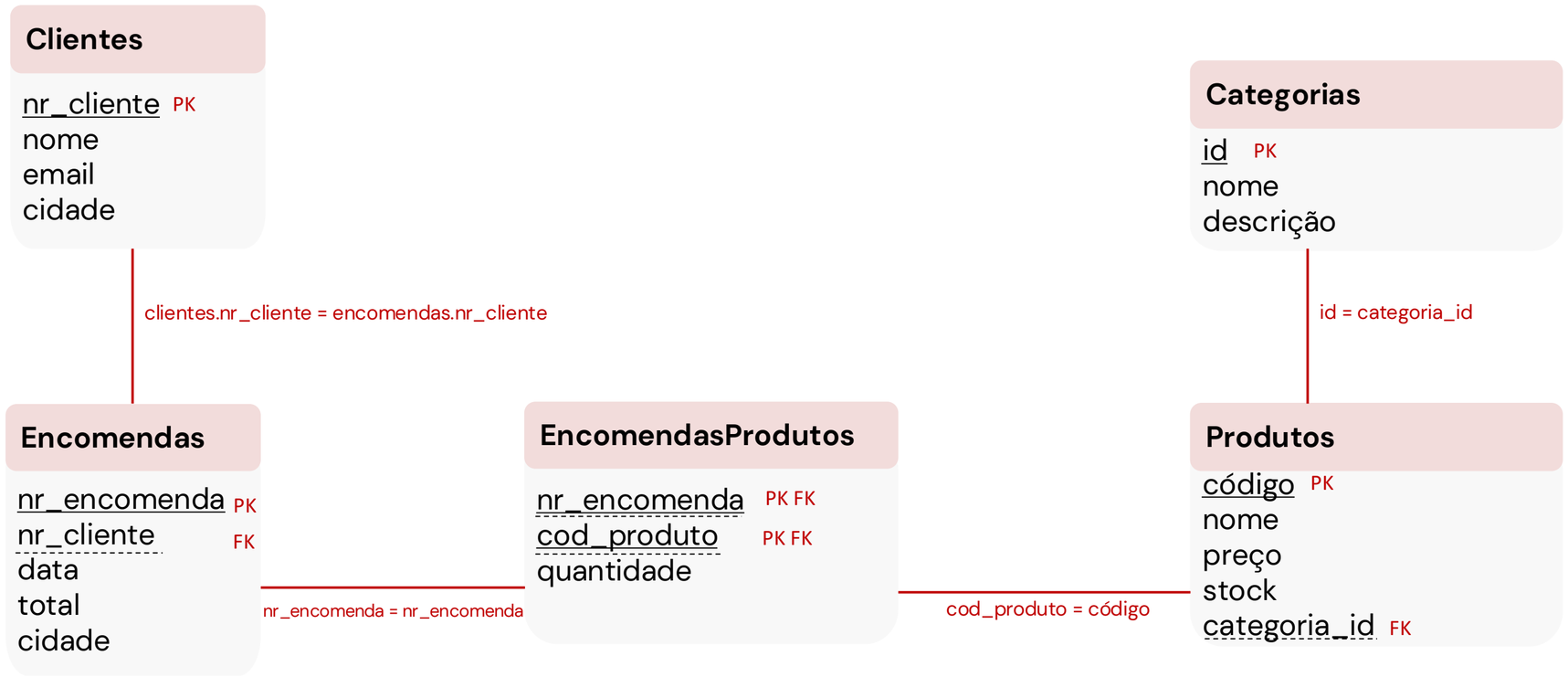
$\times$	cartesian product
$\cup$	union
$\cap$	intersection
$-$	difference
$\bowtie$	join
$\div /$	division/quotient

– Relational Operators: Selection (σ), Projection (π) e Join ( ).
– Operators of Sets: Union ( ), Intersection (∩), Difference (-), Division (/) e Cartesian Product (×).

Relational Algebra

Case Study

Database Schema (simplified)



Relational Algebra

Case Study

The Database (simplified)

Clientes

nr_cliente	nome	email	cidade
1000	Rita	rita@exemplo.pt	Braga
2000	Ana	ana@exemplo.pt	Guimarães
3000	Pedro	pedro@exemplo.pt	Vizela

Encomendas

nr_encomenda	nr_cliente	data	total	cidade
10	1000	20/01/2024	16.10	Braga
11	2000	11/02/2024	51.98	Guimarães
12	3000	07/07/2025	10.35	Guimarães
13	2000	12/12/2025	25.99	Fafe

Produtos

código	nome	preço	stock	categoria_id
1111	Rádio	10.35	10	EL
2222	Calças de ganga	25.99	20	RO
3333	Atum	1.15	30	CO
4444	iPad Air	350.10	5	EL

Categorias

id	nome	descrição
EL	Eletrónico	Inclui telemóveis e computadores.
RO	Roupa	Inclui peças de criança e adulto.
CO	Comida	Inclui todo o tipo de comida.

EncomendasProdutos

nr_encomenda	cod_produto	quantidade
10	1111	1
10	3333	5
11	2222	2
12	1111	1
13	2222	1

Relational Algebra

Operators

Selection (σ)

The selection operation works on a single relation R and defines a relation that contains only the tuples of R that satisfy the condition, i.e.:

- Selects tuples (lines) of a relationship that satisfy a condition;
- It works like a filter;
- It keeps only the data that meets the condition.



(a) Selection

$$\sigma_{condition}(R)$$

σ → Selection operator
R → Ralation (table)

The result has the same attributes as R

Conditions
- Comparisons: =, <, ≤, >, ≥, ≠
- Can use:
  - AND (□)
  - OR (□)
  - NOT (¬)

Relational Algebra

Operators

Selection (σ)

Considering the database presented above.

Example 1: What are the customer 2000 orders?

$$\sigma_{\text{nr_cliente}=2000(}\text{Encomendas})$$

nr_encomenda	nr_cliente	data	total	cidade
11	2000	11/02/2024	51.98	Guimarães
13	2000	12/12/2025	25.99	Guimarães

Example 2: Which products are priced over €10 and stock under 30?

$$\sigma_{\text{preço}>10 \,\wedge\, \text{stock}<30}(\text{Produtos})$$

código	nome	preço	stock	categoria_id
2222	Calças de ganga	25.99	20	RO

Example 3: Who are Braga's customers?

$$\sigma_{\text{cidade} = \text{'Braga'}}(\text{Clientes})$$

nr_cliente	nome	email	cidade
1000	Rita	rita@exemplo.pt	Braga

Relational Algebra

Operators

Projection (π)

The projection operation works on a single relation R and defines a relation that contains a vertical subset of R, extracts the values of the specified attributes (columns) eliminating duplicates, i.e. the relation resulting from a projection operation does not contain repeated records.



(b) Projection

$$\pi_{attributes}(R)$$

π → projection operator
R → relation (table)

The result contains only the chosen attributes.

Relational Algebra

Operators

Projection (π)

Considering the database presented above.

Example 1: What are the numbers of the customers who have placed orders?

$$\pi_{\text{nr_cliente}}(\text{Encomendas})$$

nr_cliente
1000
2000
3000

Example 2: What are the names and emails of the customers?

$$\pi_{\text{nome, cidade}}(\text{Clientes})$$

nome	email
Rita	rita@exemplo.pt
Ana	ana@exemplo.pt
Pedro	pedro@exemplo.pt

Relational Algebra

Operators

Selection (σ) e Projection (π)

Considering the database presented above.

Example 1: What are the numbers and dates of orders placed in the year 2024?

$$\pi_{\text{nr_encomenda,data}}(\sigma_{\text{YEAR(data)}=2024}(\text{Encomendas}))$$

nr_encomenda	data
10	20/01/2024
11	11/02/2024

Example 2: What is the current stock of Atum?

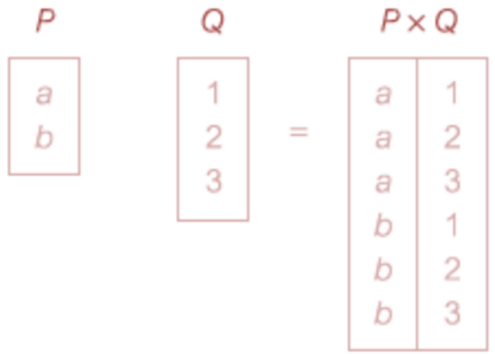
$$\pi_{\text{stock}}(\sigma_{\text{nome}='Atum'}(\text{Produtos}))$$

nome	email
Rita	rita@exemplo.pt
Ana	ana@exemplo.pt
Pedro	pedro@exemplo.pt

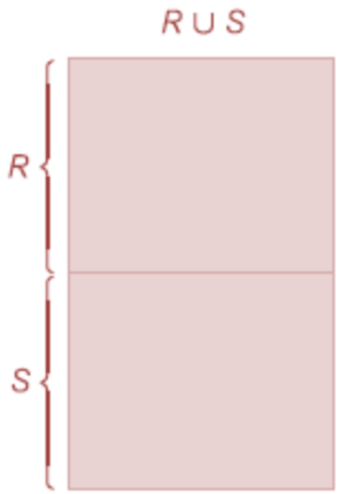
Relational Algebra

Operators

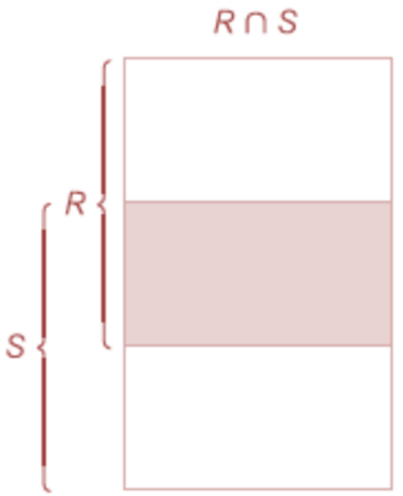
Sets



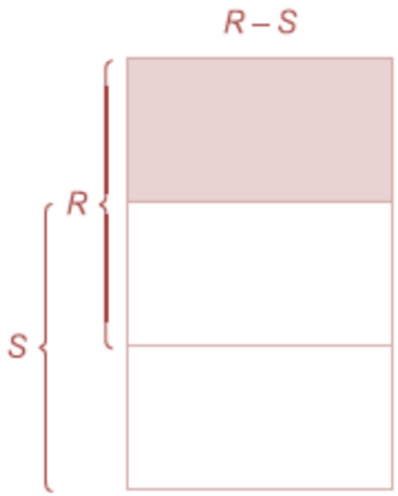
(c) Cartesian product



(d) Union



(e) Intersection



(f) Set difference

Relational Algebra

Operators

Union (∪)

$R \cup S$

The union of two relations R and S defines a relation that contains all tuples that are in R, or in S, or in both R and S, with duplicate tuples removed. R and S must be union-compatible, meaning they must have the same number of attributes and compatible domains, ensuring a valid result.

Example: List all the cities where customers live or where deliveries have been made.

$$\pi_{cidade}(Clientes) \cup \pi_{cidade}(Entregas)$$

cidade
Guimarães
Braga
Vizela
Fafe

Any record that appears simultaneously in the two relations involved in the operation appears only once in the resulting relation.

Relational Algebra

Operators

Intersection (∩)

$R \cap S$ The intersection operation defines a relation that consists of the set of all tuples that are present in both R and S. R and S must be union-compatible.

Example: List all the cities where customers live and where deliveries have been made.

$$\pi_{cidade}(Clientes) \cap \pi_{cidade}(Entregas)$$

cidade
Braga
Guimarães

Relational Algebra

Operators

Difference (−)

$R - S$ The difference operation defines a relation that consists of the tuples that belong to relation R but are not in S. R and S must be union-compatible.

Example: List all the cities where customers live but no deliveries have been made.

$$\pi_{cidade}(Clientes) - \pi_{cidade}(Entregas)$$

cidade
Vizela

R ∩ S = R − (R − S)

Relational Algebra

Operators

Cartesian Product ($\times$)

$R \times S$

The Cartesian product operation defines a relation that is the concatenation of each tuple of R with each tuple of relation Q. The schema of the resulting relation contains all the attributes of R and Q, presented in the order in which they appear in R and Q, respectively.

Example

$$(\pi_{nome,nif}(Clientes)) \times (\pi_{nome,nif}(Carros))$$

Clientes

nome	nif
Daniel	1234
Carlos	2233
Diego	9908
Robert	3276

X

Carros

carro	nif
Bmw	1234
Ferrari	9908

=

nome	clientes.nif	carro	carros.nif
Daniel	1234	Bmw	1234
Daniel	1234	Ferrari	9908
Carlos	2233	Bmw	1234
Carlos	2233	Ferrari	9908
Diego	9908	Bmw	1234
Diego	9908	Ferrari	9908
Robert	3276	Bmw	1234
Robert	3276	Ferrari	9908

Relational Algebra

Operators

Cartesian Product ($\times$)

Example

The relation from the previous example contains extra information. For example, the second tuple contains different NIF values. To obtain the desired result, we need to perform a selection on this relation to extract the tuples where clientes.nif = carros.nif.

$$\sigma_{clientes.nif\ =\ carros.nif}((\pi_{nome,nif}(Clientes)) \times (\pi_{nome,nif}(Carros)))$$

nome	clientes.nif	carro	carros.nif
Daniel	1234	Bmw	1234
Diego	9908	Ferrari	9908

Relational Algebra

Operators

Attribution (←)

The assignment operation allows the result of a relational algebra query expression to be assigned to a new relation.
For example, when we present the following expression:

$$R \leftarrow (\sigma_{\text{cidade} = \text{'Guimarães'}}(\text{Encomendas}))$$

we want R to be associated with the result of the expression.

R

nr_encomenda	nr_cliente	data	total	cidade
11	2000	11/02/2024	51.98	Guimarães
12	3000	07/07/2025	10.35	Guimarães

$$S \leftarrow (\pi_{nr_encomenda,total}(R))$$

S

nr_encomenda	total
11	51.98
12	10.35

Relational Algebra

Operators

Renaming (ρ)

$$\rho_S(E)$$

ou

$$\rho_{S(a_1,a_2,\ldots,a_n)}(E)$$

The rename provides a new S name for the expression of E, and optionally the attribute names such as a1, a2,. . . , an.

Relational Algebra

Join Operations

The join operation is used to combine information from two or more tables. A join can be defined as a Cartesian product followed by selection and projection operations.

Inner joins

$\bowtie$	Natural join
$\bowtie_F$	Theta-join
$\bowtie_{A=B}$	equi-join
$\ltimes$	semi-join

Outer joins

$⟕$	Left outer join
$⟖$	Right outer join
$⟗$	Full outer joint

Relational Algebra

Join Operations

Natural Join (⋈)

The Natural Join combines two relations based on common attributes with equal values, performing a Cartesian product, filtering the matching records, and eliminating duplicate attributes, resulting in a new relation.

Example: Which customers placed orders?

$$Encomendas \bowtie Clientes$$

nr_encomenda	nr_cliente	data	total	cidade	nome	email	cidade
10	1000	20/01/2024	16.10	Braga	Rita	rita@exemplo.pt	Braga
11	2000	11/02/2024	51.98	Guimarães	Ana	ana@exemplo.pt	Guimarães
12	3000	07/07/2025	10.35	Guimarães	Pedro	pedro@exemplo.pt	Vizela
13	2000	12/12/2025	25.99	Fafe	Ana	ana@exemplo.pt	Guimarães

Relational Algebra

Join Operations

Theta-Join ($\bowtie_F$)

The theta join combines two relations by selecting from the Cartesian product only the tuples that satisfy a comparison condition between attributes of each relation (such as =, ≠, <, ≤, >, ≥). This operation can be seen as a Cartesian product followed by a selection, where the condition determines which tuples remain in the result.

$$R \bowtie_F S = \sigma_F(R \times S)$$

Relational Algebra

Join Operations

Equi-Join ($\bowtie_{A=B}$)

The equi-join is a special case of the theta join in which the condition uses the equality operator (=), allowing the matching of equal attributes from two relations.

Example: What are the names of the product categories that exist with stock greater than 15?

$$(\sigma_{stock>15}(Produtos)) \bowtie_{produtos.categoria_{id}=categorias.id} \text{Categorias}$$

ou

$$\text{P} \leftarrow (\sigma_{stock>15}(Produtos))$$
$$\text{R} \leftarrow \text{P} \bowtie_{produtos.categoria_id=categorias.id} \text{Categorias}$$

código	produtos.nome	categoria_id	categoria.nome
2222	Calças de ganga	RO	Roupa
3333	Atum	CO	Comida

Relational Algebra

Join Operations

Semi-join ($\ltimes_{A=B}$)

The semi-join operation defines a relation that contains the tuples of R that participate in the join of R with S. In other words, it returns only the rows from the first table for which there is a matching row in the second table.

$$- \; R \ltimes S = \pi_{r1,\ldots,rn}(R \bowtie S)$$

Example: What are the details of customers who placed orders in 2024?

$$Clientes \ltimes_{clientes.nr_cliente=encomendas.nr_cliente} (\sigma_{YEAR(data)=2024}(Encomendas))$$

nr_cliente	nome	email	cidade
1000	Rita	rita@exemplo.pt	Braga
2000	Ana	ana@exemplo.pt	Guimarães

Álgebra Relacional

Join Operations

Outer Join

The outer join operation allows two relations to be combined while also preserving the records that do not have a match in the other relation. Unlike the natural join, it does not eliminate these records, instead filling the missing attributes with NULL values. There are three main types:

- Left outer join (□), which preserves all records from the left relation;
- Right outer join (□), which preserves all records from the right relation;
- Full outer join (□), which preserves all records from both relations.

Álgebra Relacional

Operações de Junção

Left outer join (□)

The left outer join operation includes in the final relation all records from the left relation, even when they do not satisfy the defined join conditions.

Example: Knowing which orders each product has (or doesn't have).

$$(Produtos) ⟕_{produtos.codigo=encomendas.cod_produto} (\pi_{nr_encomenda}(\text{Encomendas}))$$

código	nome	preço	stock	categoria_id	nr_encomenda
1111	Rádio	10.35	10	EL	10
1111	Rádio	10.35	10	EL	12
2222	Calças de ganga	25.99	20	RO	11
2222	Calças de ganga	25.99	20	RO	13
3333	Atum	1.15	30	CO	10
4444	iPad Air	350.10	5	EL	NULL

Registration without equivalent in the right-hand side table.

When there are no corresponding values, they are null.

Relational Algebra

Join Operations

Right outer join (⟖)

The right outer join operation, unlike the left outer join, includes in the final relation all records from the right relation, even when they do not satisfy the defined join conditions.

Full outer join (□)

The full outer join operation includes in the final relation all records from both the left and the right relations, even when they do not satisfy the defined join conditions.

Relational Algebra

Operator

Division

$R \div S$

The division operation defines a relation over the attributes C, consisting of the set of tuples in R that are associated with every tuple in S.

$$T_1 \leftarrow \Pi_C(R)$$
$$T_2 \leftarrow \Pi_C((T_1 \times S) - R)$$
$$T \leftarrow T_1 - T_2$$

Example: Identify all customers who viewed all three-bedroom properties.

$$(\Pi_{clientNo,\ propertyNo}(Viewing)) \div (\Pi_{propertyNo}(\sigma_{rooms\ =\ 3}(PropertyForRent)))$$

$\Pi_{clientNo,propertyNo}(Viewing)$

clientNo	propertyNo
CR56	PA14
CR76	PG4
CR56	PG4
CR62	PA14
CR56	PG36

$\Pi_{propertyNo}(\sigma_{rooms=3}(PropertyForRent))$

propertyNo
PG4
PG36

RESULT

clientNo
CR56

Relational Algebra

Operator

Aggregation/Grouping Operations (γ)

$\gamma_{AL}(R)$

The aggregation operation (γ) allows us to group the tuples of a given relation according to one or more attributes of that relation, based on a specified grouping criterion.
A list of aggregation functions, AL, is applied to relation R, defining a new relation over the aggregated list. AL contains one or more pairs of the form (attribute, function).

The main aggregation functions are:

- COUNT – Returns the number of values of the associated attribute.
- SUM – Returns the sum of the values of the associated attribute.
- AVG – returns the average of the associated attribute values.
- MIN – Returns the smallest value in the associated attribute.
- MAX – Returns the highest value in the associated attribute.

Relational Algebra

Operator

Aggregation/Grouping Operations (γ)

Example 1: How many products cost more than €10?

$$\gamma_{\,contagem \leftarrow COUNT(codigo)}(\sigma_{preço>10}(Produtos))$$

contagem
3

Example 2: What is the total amount spent by each customer?

$$\gamma_{nr_cliente;SUM(total) \rightarrow total_gasto}(Encomendas)$$

cod_produto	soma
1000	16.10
2000	77.97
3000	10.35

Relational Algebra

Operator

Ordering (τ)

The ordering operation consists of organizing the tuples of a relation according to one or more attributes, in either ascending (ASC) or descending (DESC) order, without altering the content or the schema of the relation, only its presentation.

Example 1: Sort customers alphabetically (name)

$$\tau_{nome\ ASC}(Clientes)$$

nr_cliente	nome	email	cidade
2000	Ana	ana@exemplo.pt	Guimarães
3000	Pedro	pedro@exemplo.pt	Vizela
1000	Rita	rita@exemplo.pt	Braga

Example 2: Sort the products in such a way as to present the most expensive first.

$$\tau_{preço\ DESC}(Produtos)$$

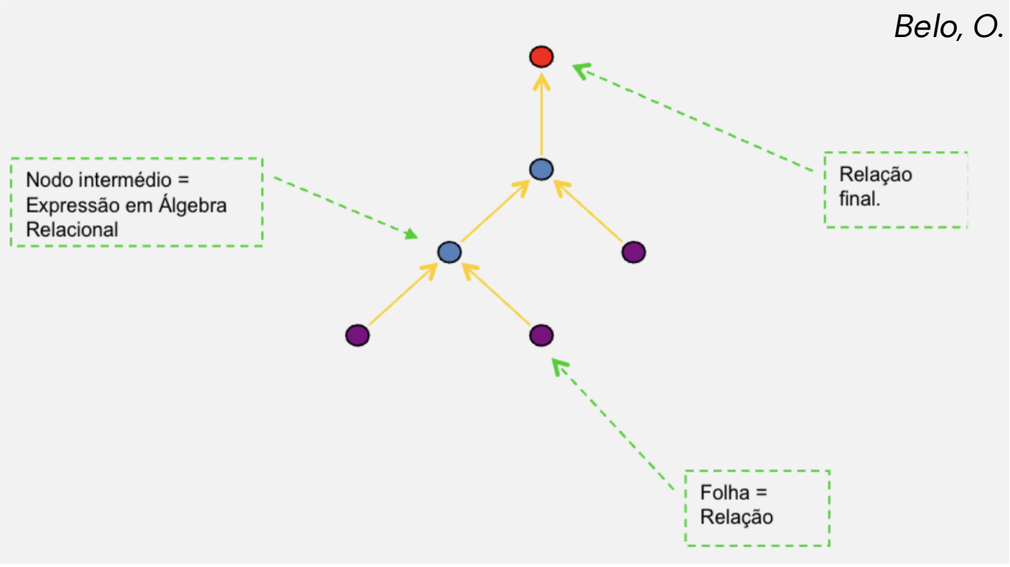
código	nome	preço	stock	categoria_id
4444	iPad Air	350.10	5	EL
2222	Calças de ganga	25.99	20	RO
1111	Rádio	10.35	10	EL
3333	Atum	1.15	30	CO

Relational Algebra

Query Trees

Query trees are a fundamental tool for representing and analyzing expressions in Relational Algebra, allowing all stages of their processing and the use of relations throughout the solution to be visualized. In these trees, the leaves represent relations, the intermediate nodes represent relational algebra operations, and the final node corresponds to the resulting relation.

To make it easier to draw the trees and write the expressions, it is common to invert the tree, placing the base relations (the leaves) at the top of the page.

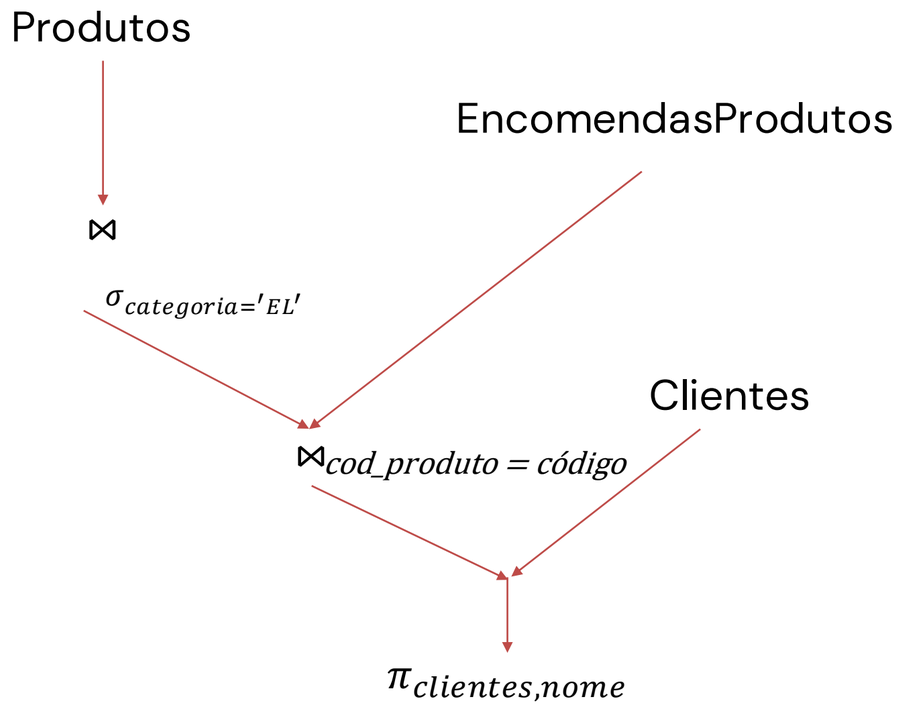


Relational Algebra

Example

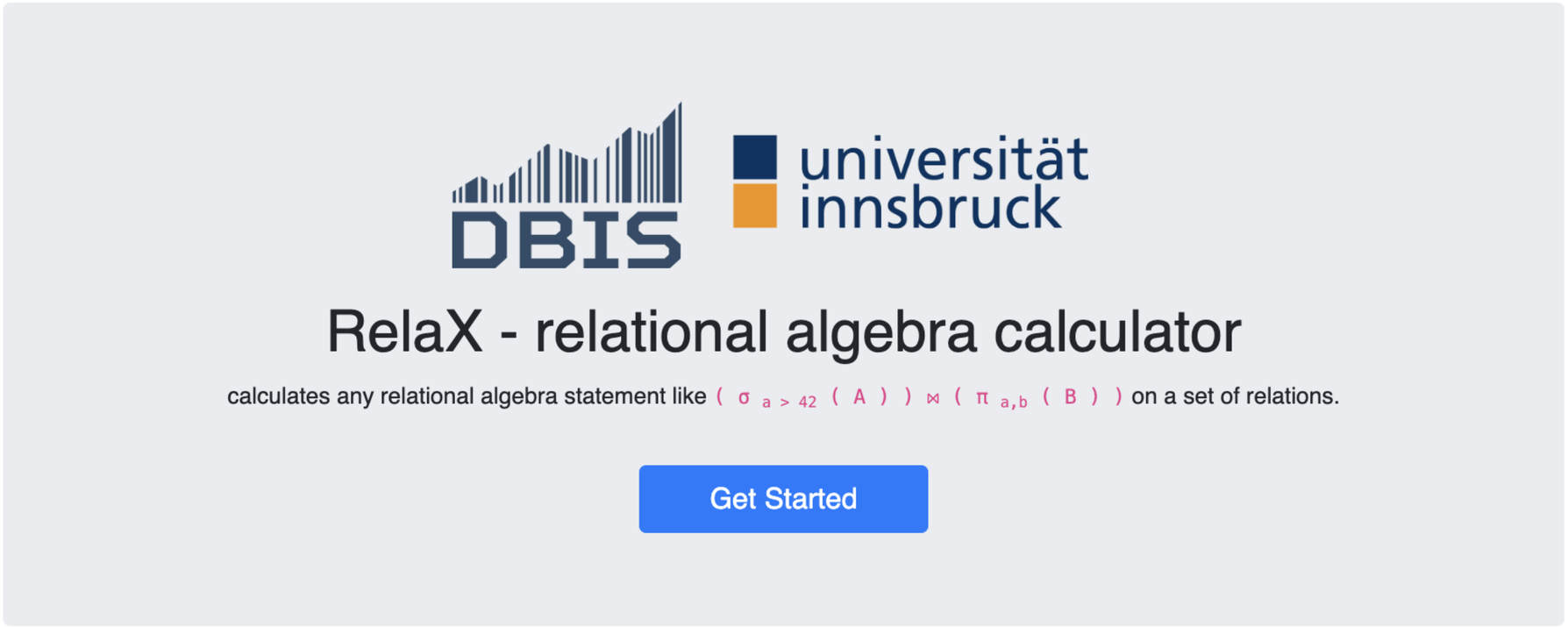
What are the numbers of the customers who bought products from the 'EL' category?

$$\pi_{encomendas.nr_client}\ (\sigma_{categoria='EL'}\ (Produtos) \bowtie_{cod_produto\ =\ código}\ (EncomendasProdutos \bowtie Encomendas))$$



Relational Algebra

RelaX – Relational Algebra Calculator



http://dbis-uibk.github.io/relax/

Relational Algebra

References

- Connolly, T., Begg, C., Database Systems: A Practical Approach to Design, Implementation, and Management, 6th edition, Pearson, January, 2014.
- Garcia-Molina, H., Ullman, J., Widom, J., Database Systems: The Complete Book, 2nd Edition, Pearson, June, 2008.
- Date, C., An Introduction to Database Systems, 8th Edition, Pearson, July, 2003.
- Molková, L., Theory and Practice of Relational Algebra, LAP Lambert Academic Publishing, January, 2012.

Universidade do Minho

Ano Letivo: 2025/26

Databases

Relational Algebra



António Abelha | abelha@di.uminho.pt